Automated Planning Report

Dario Tortorici
Matriculation number 248443
DISI, University of Trento
Trento, Italy
Email: dario.tortorici@studenti.unitn.it

January 31, 2024

I. Introduction

The aim of this report is to provide a comprehensive overview of the resolution of the automated planning course assignment. The assignment consisted of five exercises, either in a medical scenario or by choice. For this assignment, I chose to implement the healthcare scenario to evaluate different domains and problems, analyzing how modeling choices impact overall performance. In summary, the healthcare scenario involves multiple units within a single healthcare facility, each specializing in supplying medical supplies to medical staff or escorting patients to their designated rooms.

A. Modeling choices and assumptions

Throughout the process, I encountered challenges related to the modeling and compatibility across various planners. To address these issues, I adopted a modeling approach that strictly avoids the use of Action Description Language (ADL), ensuring that no negative preconditions are used. Instead, I explicitly represented dualities through separate predicates, with each action affecting these predicates ensuring both are consistently updated. This approach guarantees the model's portability across all planners. Additionally, I integrated the "delivery" and "empty a box" requirements into a single action. This is in accordance with the assignment's stipulations, since the "unload" action is written to be executed in a manner that ensures the medical unit acquires the specified content. Alternatively, creating a separate unboxing action that allows the bot to take out the content without delivering it led to complications, including infinite loops with some planners, making it impossible to find a solution. As a result, I assumed that unboxing could only occur as part of the delivery process. As for the other robot action requirements, I created a specific action in the domain for each of them.

II. Exercise 1

A. Domain definition

For the first problem, I began with a simplified definition of the domain, called `healthcare_scenario_1`, and made significant assumptions to streamline the model. In this preliminary approach, the box, its contents, and the patient were designated as 'non-pickable' objects, meaning that they did not retain a location after an action that assigned them to other objects. However, the final location was regained after this attachment was lost. It was also assumed that the bot could load an infinite number of boxes.
To explore the trade-offs of a more robust system, I refined the domain in a second version, called `healthcare_scenario_1_rigorous` by introducing additional predicates and constraints. These refinements included tracking the availability of specific content (`content_available`), enabling the bot to transport a box and unload it at intermediate locations without requiring immediate delivery to the final destination

(`move_bot_with_box`). Furthermore, I modeled the content as singular types to ensure alignment with the requirement that medical units do not consider the quantity of a given item (`medic_has` was substituted with `has_scalpel` and the corresponding predicates for the other contents, where the medic is the argument). Finally, I restricted the system to handling only one box at a time `bot_has_box`, bypassing the need to account for box capacity and multiple items within a single container `box_has_content`. The same logic was applied to units, that can escort one patient at the time with the predicate `unit_busy`. This domains were tested through two progressively complex problems, each increasing the number and variety of modeled objects, to evaluate the impact of these modifications on performance and planning outcomes.

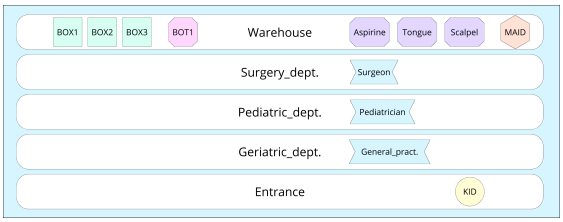

Fig. 1. Visual representation of the hospital layout in `problem_1_scenario_1`.

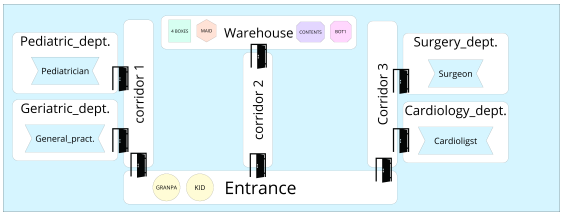

Fig. 2. Visual representation of the hospital layout in `problem_2_scenario_1`.

B. Problem Definition

1) First Problem: In the first problem, the scenario is configured with a single bot, one maid, one patient, one instance each of three types of content, three medics (each requiring a specific type of content) and three boxes. The latter number was chosen to investigate whether the bot, which has a box for each content, might want to put everything in boxes and then move on, or whether it prefers to perform the whole cycle of actions for the deliveries sequentially. Furthermore, The hospital layout is modeled as a sequential structure consisting of the following locations: the entrance, the geriatric department (abbreviated as `dept.` in the code), the pediatric department, the surgery department, and the warehouse. Each location is connected to its neighbors, with bi-directional accessibility throughout the departments.

2) Second Problem: In the second problem, I added more structure to the hospital. I added corridors where there are several departments to be visited, to better resemble the structure of a real hospital. I also added a new instance of everything: box, content, department, doctor and patient to make the problem more complex.

C. Problem Evaluation

1) Simple Domain: In the domain `healthcare_scenario_1` and problem `problem_1_scenario_1`, the Fast Forward (FF) planner used Best-First Search (BFS) to identify a valid plan comprising 25 steps to achieve the defined goals. The plan followed the expected sequence: the bot loaded all the boxes together, filled them, and moved through the hospital layout, unloading at the required locations. The computational time was negligible, clocking below 0.00 seconds.

2) Rigorous Domain:

a) First Problem: **Fast Forward.** In the second domain, FF and BFS were again employed to compare results with the simpler domain. The planner found a valid plan with 29 steps, four additional steps compared to the simpler domain. These extra steps were due to the stricter rule that the bot can load only one box at a time. The bot reused the same box for deliveries, loading, filling, and delivering iteratively.

The primary difference between the two domains lies in the complexity of the planning process. The number of literals increased from 47 to 86, and

the stricter domain required evaluating 301 states (compared to 101 in the simpler domain). Despite this increase, the overhead remained acceptable, with no measurable difference in computational time (still below 0.00 seconds). To further analyze performance, I tested a larger problem.

Fast Downward. The Fast Downward planner executed the planning task generating 291 relevant atoms, generating 403 auxiliary atoms to support the planning process. Moreover, the planner generated 105 rules and completed the planning task in 0.060 seconds of CPU time and 0.070 seconds of wall-clock time. It is slightly more than ff, but not by a significant margin.

Optic. Using BFS with WA*, the temporal planner Optic evaluated 1,034 states, more than FF due to its consideration of time intervals, action overlaps, and scheduling, even in classical problems. The solution that was found was a 30-step solution, which is one step longer than the solution that FF found. This is due to the fact that FF's Relaxed Plan Graph (RPG) heuristic is optimized for classical planning.

b) Second Problem: **Fast Forward.** FF successfully resolved the second issue in 51 steps, with a response time of 0.00 seconds.

Fast Downward. The Fast Downward planner executed the larger problem generating 606 relevant atoms and completed the planning task in 0.120s seconds of CPU time and 0.114 seconds of wall-clock time.

Optic. The planner took 2.93 seconds to complete because it first found an immediate solution and then refined it to generate a more time-optimal second plan.

III. Exercise 2

A. Domain definition

In the second exercise, I have added a new action that operates on carriers, and modified some of the previous actions to ensure they operate on carriers properly. I have modeled the carriers as separate objects from the bot. Therefore, from now on, every delivery bot will need to have carriers in order to accomplish the task. This distinction enables the placement of carriers in diverse locations and facilitates the utilization of different carriers by different bots. This approach is logical, as it allows a unique bot to choose between carriers. For instance, it can select the one with the largest storage capacity or the nearest one. These choices enable the planner to reason in order to minimize steps. Moreover, I have modified `load_bot` and `unload_bot` to work with carriers rather than directly with boxes. Same for `move_bot_with_box` that became `move_bot_with_carrier`. In particular, the delivery cycle now requires the box to be unloaded from the carrier before the contents are delivered and then reloaded into the carrier before the empty box is moved. This choice was made because I imagine the carrier as a parcel trolley in which boxes can be stacked.

In the domain scope, two different instances have been created. The first domain uses capacity spaces as objects, while the second one instead uses the `:numeric-fluents` approach. This latter approach offers greater flexibility for mapping high numbers, although not all planners provide support for this requirement.

B. Problem definition

1) Object approach: The capacity spaces are defined in the problem file, and it is necessary to declare an object for each space in order to maintain a record of load.

2) Numeric fluents approach: In the `:numeric-fluents` approach, two functions were defined, `carrier_capacity` and `carrier_load` to track respectively capacity and current loads of carriers. In this modeling version, the action `load_carrier` checks whether the capacity is less than the actual load. If this is the case, the action increases the load. Conversely, in the unload process, the action functions in the inverse manner. Specifically, the unload process is initiated only when a box has been carried, and then the load is decreased.

3) First Problem: In both domains, the carrier object is initialized with its attributes, i.e. location, load (initially set to 0), and capacity. The capacity is defined on a one-to-one basis, binding each capacity object directly to a specific carrier object. For simplicity, only one carrier is mapped in the

predefined simple problem, with a capacity set to 2. This limited number has been chosen to prevent the bot from taking all three boxes simultaneously, thus providing a more interesting solution.

4) Second Problem: For both domains, two carrier objects have been initialized. Each carrier's capacity is explicitly bound using a one-to-one relationship, defined by the predicate `carrier_has_capacity`. The first carrier retains the same capacity as in the previous problem (2 boxes). The second carrier is given a higher capacity of 4 boxes. This setup was designed to observe whether the bot would prioritize the more spacious carrier when performing tasks.

C. Problem evaluation

1) Standard Domain:

a) First Problem: **Fast Forward.** The total time for the process was 0.00 seconds, with 30 steps. This is one more than the previous plan in the first exercise, although it can and does carry two boxes at a time because it includes the attachment action `attach_carrier` and the transitory actions `unload_carrier` and `load_carrier` on each delivery.

Fast Downward. The solution was established in 0.070 seconds CPU and 0.072 seconds wall-clock time.

b) Second Problem: **Fast Forward.** The planner found a solution of 54 steps in 0.14 seconds. In comparison to Exercise 1, this exercise involved three additional steps. The planner preferred to fill three boxes, load the two boxes containing the items to be delivered in the same corridor, and deliver them. The planner then returned to the warehouse to retrieve the other two boxes of content, which were to be delivered in a different corridor.

Fast Downward. The solution was established in 0.130s seconds CPU and 0.136s seconds wall-clock time.

2) :numeric-fluents Domain:

a) First Problem: **Optic.** I have evaluated this domain exclusively with Optic, as FF and downward do not support numeric fluents. The solution was found instantly, but there are 37 steps.

b) Second Problem: **Optic.** Again, the solution was found immediately, but there are 63 steps, considerably more than the corresponding ff counterpart with capacity spaces as objects.

IV. Exercise 3

A. Domain definition

At a high level, tasks such as delivering medical supplies or escorting patients are defined. These tasks are further broken down into specific subtasks. For example, the task of delivering an item to a medic involves moving the carrier to the required location, loading the required content onto the carrier, moving the carrier to the medic's location and delivering to him the content. Each sub-task is supported by alternative methods to handle different contexts and ensure robustness of the solution. Movement-related tasks include strategies for recursive and direct movement, depending on the situation. Additionally, safeguards like fallback methods and `noop` actions are implemented to ensure ending of recursion.

For loading and filling the box the `m_attach_carrier_with_content` method is used to load a pre-filled box into the carrier. Instead, the `m_load_after_unload` method handles next scenarios where the carrier has already loaded an empty box. More specifically, the method allows you to unload a previously carried empty box and then reload it once it has been refilled. To streamline resolution time, I modeled the ordering of the independent tasks, which simplifies the solution but limit flexibility. Additionally, only for this exercise, it is assumed that the carrier is already attached to the bot at the start of the process, further optimizing execution time.

B. Problem definition

The definition of the problem is the same as the previous exercise, but using tasks as goals instead of predicates.

1) Third Problem: In order to evaluate the reason for the long solution time of the second problem, a new intermediate problem with a simplified health-care facility was created. In particular, each location is the same as in the first problem, but with the addition of the cardiology department, sequentially connected without corridors. Furthermore, all the other instances present in problem 2 are recreated in problem 3, except, as already mentioned, the corridors.

C. Problem evaluation

1) First Problem: The solution found by Panda has 36 steps. However, some of these actions are not relevant to achieving the goal, due to the modeling choices. For instance, the solution begins with a no-op operation. Furthermore, since the sub-task reloads the carrier with the empty box after the delivery of the box, the planner performs this action even though it is not required.

2) Second Problem: Panda couldn't solve the problem in acceptable time, so I had to stop the execution.

3) Third Problem: Panda successfully formulated a plan, indicating that the inability to solve the second problem is not due to the number of objects, but rather the complexity of the location and the range of possible movements.

V. Exercise 4

A. Domain definition

The domain incorporates now `:durative-actions`, enabling temporal actions with specified duration. Additionally, the bot and the unit has now the predicate idle state, which prevents overlapping actions that cannot occur simultaneously.

The durations for these actions are arbitrarily assigned, using 1 as the base unit of time. These durations are modeled based on what I consider to be realistic differences between tasks. For example, moving without a carrier is assumed to be faster than moving with a carrier. Similarly, a unit escorting a patient (with maybe mobility problems) is expected to take more time than normal movement. In particular, the movement of bots (unit and delivering bots) is modeled to take 2 time units to move alone and 3 with patient or carrier. Filling the box is expected to be more time consuming than delivering, respectively 3 and 2 time units, while for loading and unloading the box into carriers is supposed to take 3 units of time due to the delicacy required. The other actions are modeled as "snap actions", so they only require 1 time unit.

B. Problem definition

The problems are identical and have been formulated in exactly the same way as exercise 2, version with capacity objects.

C. Problem Evaluation

1) First Problem:

a) POPF: POPF successfully found a solution immediately, with a plan execution time of 75 time units.

b) OPTIC: Optic successfully identified two solutions immediately. The second solution was significantly superior to his first and the POPF solution. It was able to generate a plan that could be executed in 58 time units. It has been noted that the delivering bot has an unusual behavior. At the start of the plan, the bot enters surgery dept. and then returns to the warehouse before performing the necessary logical actions.

c) Temporal Fast Downward: Temporal Fast Downward generated three solutions immediately, with the best solution having an execution time of 76 time units. Although it was slightly slower than POPF, the difference in performance is significant with OPTIC proposed solution.

2) Second Problem:

a) POPF: POPF took a few seconds to run, effectively pruning unnecessary actions such as loading and unloading the carrier when it identified the use of another carrier. This optimization allowed it to find a solution with a plan execution time of 133 time units.

b) OPTIC: OPTIC was unable to generate a solution to this problem within an acceptable time, so it was aborted.

c) Temporal Fast Downward: Temporal Fast Downward, as OPTIC, was unable to find a solution in a reasonable amount of time.

VI. Exercise 5

A. Domain definition

In the fifth exercise, I used the domain of the fourth exercise. I matched the duration of the PDDL actions in the '.cpp' files by introducing it as a parameter through the 'launch.py' file. This way, I could maintain a more structured approach and avoid the need to modify each '.cpp' file individually every time the duration of the PDDL action changes. This also added simplicity to the process. Given that the '.cpp' files are essentially wrappers, I choosed to operate each 'ActionExecutorClient' at a fixed 100ms rate, so I could use a singluar formula to ensure consistency in the 'do_work()' function every time the action is executed.
The formula used to update the progress is calculated as follows:

$$\text{progress\_} \mathrel{+}= \frac{1}{\text{duration} \times 10}$$

Where 'duration' is the duration of the PDDL action passed from the 'launch.py' file as mentioned above.

B. Problem definition

The problems were analogous to those in exercise 4. However, they have been adapted to be written using the ros2 terminal syntax and stored in the launch folder to direct sourcing through the terminal.

C. Problem Evaluation

1) Problem 1: Ros2 POPF planner successfully created and executed a plan of 54 timesteps, removing the additional steps found in the OPTIC planner during exercise 4.

2) Problem 2: As was the case in exercise 4, even the Ros2 POPF planner was unable to solve the problem.